

05_Architecture.md

MonitorAI v1.5 Architecture Specification

Version: 1.5

Status: Baseline

1. Purpose

This document defines the technical architecture of MonitorAI v1.5.

The architecture must support:

- Standardized image acquisition
 - Real-time guidance
 - Metadata collection
 - Offline operation
 - FileMaker integration
 - Future AI dataset expansion
-

2. Architecture Philosophy

MonitorAI must be:

- Modular
- Testable
- Maintainable
- Extensible

The application should avoid tightly coupled logic.

Clinical workflow logic must be separated from UI rendering.

3. Architecture Pattern

Required Pattern:

MVVM

Supporting Patterns:

- Service Layer
 - Repository Pattern
 - Dependency Injection
 - Protocol-Oriented Design
-

4. Layer Structure

App Layer

↓

Feature Layer

↓

ViewModel Layer

↓

Service Layer

↓

Repository Layer

↓

API Layer

↓

FileMaker Server

5. App Layer

Responsibilities:

- App entry point
- Dependency registration
- Global navigation
- App configuration

Core Files:

- MonitorAIApp.swift
 - AppCoordinator.swift
 - DependencyContainer.swift
 - AppConfiguration.swift
-

6. Feature Layer

Major Features:

- Authentication
- Patient Selection
- Position Calibration
- Capture Workflow
- Review
- Upload
- Settings

Each feature should have its own View and ViewModel.

7. View Layer

Responsibilities:

- Render UI
- Display live camera preview
- Display overlay guide
- Display PTG rail
- Display status messages

Views must not contain:

- Business logic

- API calls
 - FileMaker logic
 - Image scoring logic
-

8. ViewModel Layer

Responsibilities:

- UI state management
- Workflow state
- User action handling
- Service coordination

ViewModels must be testable.

ViewModels should depend on protocols, not concrete implementations.

9. Service Layer

The Service Layer contains business logic.

Required Services:

- CameraService
 - PositionCalibrationService
 - OverlayGuideService
 - PTGService
 - VisibilityScoringService
 - CaptureValidationService
 - AutoCaptureService
 - PlateauDetectionService
 - VoiceGuidanceService
 - HapticFeedbackService
 - LightingService
 - MetadataService
 - StorageService
 - UploadService
 - AnalyticsService
-

10. CameraService

Responsibilities:

- Camera session setup
- Live preview
- Image capture
- Torch control
- Exposure control
- Focus control

Framework:

AVFoundation

Stored image rule:

Always 1.0x

11. PositionCalibrationService

Responsibilities:

- Detect retractor opening
- Calculate center offset
- Evaluate stability
- Trigger calibration success

Outputs:

- CalibrationOffsetX
 - CalibrationOffsetY
 - CalibrationQualityScore
 - CalibrationDurationSeconds
-

12. OverlayGuideService

Responsibilities:

- Load PNG overlay assets
- Match overlay to capture step
- Apply opacity

- Maintain aspect ratio

Overlay must not be used for scoring.

13. PTGService

Responsibilities:

- Manage PTG state
- Track Dot1–Dot6
- Trigger feedback events
- Provide PTGMaxDot

PTG should evaluate visibility, not device angle.

14. VisibilityScoringService

Responsibilities:

- Evaluate buccal visibility
- Evaluate occlusal visibility
- Provide numeric score 0–100
- Provide real-time score updates

Used by:

- PTGService
 - MetadataService
 - AutoCaptureService
-

15. CaptureValidationService

Responsibilities:

Validate:

- Focus
- Blur
- Exposure
- Composition
- Visibility

Reject unacceptable captures.

16. AutoCaptureService

Responsibilities:

- Detect capture-ready state
- Wait for stability
- Trigger capture

Rule:

$PTG \geq \text{Dot4}$

AND

Stable 4 Seconds

17. PlateauDetectionService

Responsibilities:

- Detect lack of visibility improvement
- Trigger plateau capture

Rule:

$PTG \geq \text{Dot4}$

AND

No visibility improvement

AND

Stable 4 Seconds

18. VoiceGuidanceService

Responsibilities:

- Play voice prompts
 - Prevent excessive repetition
 - Log voice events
-

19. HapticFeedbackService

Responsibilities:

- Trigger haptic events
 - Map haptics to PTG events
 - Log haptic events
-

20. LightingService

Responsibilities:

- Torch control
- Brightness monitoring
- Exposure lock
- Adaptive torch adjustment

Default torch level:

70%

Adaptive adjustment:

Allowed

Flash:

Never use

21. MetadataService

Responsibilities:

- Collect session metadata
- Collect capture metadata
- Validate metadata completeness
- Provide metadata payload for upload

Metadata must never be optional.

22. StorageService

Responsibilities:

- Local image storage
- Local metadata storage
- Offline queue persistence

Required:

No data loss

23. UploadService

Responsibilities:

- Authenticate with FileMaker
 - Upload session
 - Upload image
 - Upload metadata
 - Retry failed uploads
-

24. AnalyticsService

Responsibilities:

Track:

- Session duration

- Retakes
 - PTG distribution
 - Visibility score
 - Upload timing
 - Capture timing
-

25. Repository Layer

Repositories abstract persistence and networking.

Required:

- SessionRepository
 - CaptureRepository
 - MetadataRepository
 - UploadQueueRepository
 - SettingsRepository
-

26. API Layer

FileMakerAPIClient handles:

- Authentication
- Record creation
- Container upload
- Metadata upload
- Session completion

API layer must not contain UI logic.

27. Dependency Injection

All services must be injected.

Use protocols.

Avoid global singletons.

28. Error Handling

Errors must be:

- Typed
- Logged
- Recoverable when possible
- User-readable when necessary

No silent failures.

29. Testing Architecture

Each service must be independently testable.

Critical test targets:

- PTGService
 - VisibilityScoringService
 - AutoCaptureService
 - PlateauDetectionService
 - MetadataService
 - UploadService
-

30. Final Architecture Principle

The architecture must protect dataset quality.

If a technical shortcut reduces metadata quality, traceability, or reliability, it must not be used.